

从 0 手写 Tiny HTTP Server：零基础网络编程学习指导书

这份指导书是给“第一次接触 Linux、C++ 工程、网络编程、HTTP 服务器”的同学准备的。它不是只告诉你“看哪个文件”，而是尽量把知识点揉开讲清楚：

- 这一节要做什么。
- 要创建哪些文件夹和文件。
- 用什么 Linux 命令创建、查看、编辑、编译、运行。
- Vim 怎么进入编辑、怎么保存退出、怎么搜索代码。
- 展示的代码每一行做什么。
- 写完后怎么验证。
- 出错时怎么定位。
- 最后再对照当前项目源码。

当前项目已经实现完成。学习时建议你采用两种方式之一：

方式一：直接在当前项目里阅读、运行、修改。

方式二：新建一个练习目录，跟着本指导书从 0 写一遍，再回来看当前项目源码。

如果你是初学者，我更推荐方式二。自己从 0 敲一遍，会比只看现成代码理解快得多。

0. 学习前准备

0.1 推荐开发环境

这个项目建议在 Linux 环境中完成，例如：

Windows + Linux 虚拟机 + VS Code Remote SSH

Windows + WSL2 Ubuntu + VS Code

纯 Linux 电脑 + VS Code

云服务器 Ubuntu + VS Code Remote SSH

不建议纯 Windows + MinGW 完成整个项目，因为项目核心服务器使用 Linux 专属能力：

```
epoll
accept4
fcntl
unistd.h
Linux socket
后续可能用 sendfile
```

0.2 安装基础工具

在 Ubuntu 中执行：

```
sudo apt update
sudo apt install -y build-essential cmake gdb git curl vim tree
```

逐行解释：

- `sudo`：用管理员权限执行命令。
- `apt`：Ubuntu 的软件包管理工具。
- `update`：更新软件源索引，不是升级软件，只是刷新“软件列表”。
- `install`：安装软件包。
- `-y`：自动回答 yes，避免中途停下来询问。
- `build-essential`：包含 `gcc`、`g++`、`make` 等编译 C/C++ 的基础工具。
- `cmake`：本项目使用的构建工具。
- `gdb`：调试 C/C++ 程序的工具。
- `git`：版本管理工具。
- `curl`：发送 HTTP 请求，测试服务器很常用。
- `vim`：命令行文本编辑器。
- `tree`：用树状结构查看目录。

检查版本：

```
g++ --version
cmake --version
curl --version
vim --version
```

这些命令的作用：

- `xxx --version`：查看工具是否安装成功，以及当前版本。

- 如果提示 `command not found`，说明没有安装成功。

0.3 Linux 常用命令速查

查看当前所在目录：

```
pwd
```

解释：

- `pwd` 是 print working directory 的缩写。
- 它会输出你当前所在路径，例如 `/home/ubuntu/projects/tiny-http-server`。

查看当前目录文件：

```
ls
```

查看隐藏文件和详细信息：

```
ls -la
```

解释：

- `ls`：列出文件。
- `-l`：长格式，显示权限、大小、时间。
- `-a`：显示隐藏文件，例如 `.gitignore`。

进入目录：

```
cd tiny-http-server
```

返回上一级目录：

```
cd ..
```

创建多级目录：

```
mkdir -p include/tiny_http src tests www docs
```

解释：

- `mkdir`：创建目录。
- `-p`：如果上级目录不存在，就一起创建；如果目录已存在，也不报错。

- `include/tiny_http src tests www docs`：一次创建多个目录。

创建空文件：

```
touch CMakeLists.txt src/main.cpp
```

解释：

- `touch`：如果文件不存在，就创建空文件；如果文件存在，就更新时间戳。

查看文件内容：

```
cat CMakeLists.txt
```

解释：

- `cat`：把文件内容输出到终端。
- 适合短文件，不适合特别长的文件。

分页查看长文件：

```
less src/linux_server.cpp
```

常用按键：

- `j` 或方向下：向下滚动。
- `k` 或方向上：向上滚动。
- `/关键字`：搜索。
- `n`：跳到下一个搜索结果。
- `q`：退出。

查看目录树：

```
tree -L 3
```

解释：

- `tree`：树状显示目录。
- `-L 3`：最多显示 3 层。

查找文件：

```
find . -name "*.cpp"
```

解释：

- `find`：查找文件。
- `.`：从当前目录开始查。
- `-name "*.cpp"`：文件名匹配 `.cpp`。

搜索代码内容：

```
grep -R "HttpParser" .
```

解释：

- `grep`：搜索文本。
- `-R`：递归搜索当前目录和子目录。
- `"HttpParser"`：要搜索的关键字。
- `.`：搜索当前目录。

如果安装了 `ripgrep`，更推荐：

```
rg "HttpParser"
```

删除文件时要谨慎。本项目学习中如果必须删除，只删除一个明确文件：

```
rm parser_tests
```

不要使用：

```
rm -rf build
```

如果要清理整个构建目录，建议手动确认后再处理，或者重新创建一个新的构建目录，例如 `build-debug`。

0.4 Vim 最小生存指南

打开文件：

```
vim src/main.cpp
```

Vim 有模式。初学先记住 3 个：

普通模式：刚打开文件时所在模式，用来移动、复制、删除、搜索。

插入模式：输入文字代码。

命令模式：保存、退出、设置行号。

进入插入模式：

```
i
```

解释：

- 按 `i` 后，左下角通常会出现 `-- INSERT --`。
- 这时你可以像普通编辑器一样输入代码。

退出插入模式：

```
Esc
```

保存并退出：

```
:wq
```

解释：

- 先按 `Esc` 回到普通模式。
- 输入 `:wq`。
- `w` 表示 write 保存。
- `q` 表示 quit 退出。

只保存不退出：

```
:w
```

不保存强制退出：

```
:q!
```

显示行号：

```
:set number
```

跳到第 20 行：

:20

搜索关键字：

/handle_read

继续下一个搜索结果：

n

复制当前行：

yy

粘贴：

p

删除当前行：

dd

撤销：

u

重做：

Ctrl + r

保存文件的完整流程示例：

```
vim src/main.cpp
```

按 i

输入或修改代码

按 Esc

输入 :wq

按 Enter

如果你使用 VS Code，也可以：

```
code .
```

解释：

- `code .` 表示用 VS Code 打开当前目录。
- 如果你通过 Remote SSH 连接 Linux 虚拟机，VS Code 会直接编辑 Linux 里的文件。

第 1 课：创建项目骨架

本节目标

从空目录开始，创建一个标准 C++ 项目骨架。

完成后目录应该长这样：

```
tiny-http-server/  
├─ CMakeLists.txt  
├─ include/  
│   └─ tiny_http/  
├─ src/  
│   └─ main.cpp  
├─ tests/  
├─ www/  
└─ docs/
```

创建项目目录

如果你还没有项目目录：

```
mkdir -p ~/projects  
cd ~/projects  
mkdir tiny-http-server  
cd tiny-http-server
```

逐行解释：

- `mkdir -p ~/projects`：在用户家目录下创建 `projects`，如果已经存在也不报错。
- `cd ~/projects`：进入项目集合目录。
- `mkdir tiny-http-server`：创建本项目目录。

- `cd tiny-http-server`：进入本项目目录。

如果你已经在项目目录里，先确认位置：

```
pwd
```

创建文件夹

```
mkdir -p include/tiny_http src tests www docs
```

解释：

- `include/tiny_http`：放头文件。
- `src`：放 `.cpp` 实现文件。
- `tests`：放测试代码。
- `www`：放服务器要返回的静态网页。
- `docs`：放学习文档。

查看结果：

```
tree -L 2
```

创建入口文件

```
touch CMakeLists.txt src/main.cpp
```

打开 `src/main.cpp`：

```
vim src/main.cpp
```

输入：

```
#include <iostream>

int main() {
    std::cout << "tiny_httpd starting..." << std::endl;
    return 0;
}
```

逐行解释：

- `#include <iostream>`：引入 C++ 标准输入输出库，后面才能使用 `std::cout`。
- 空行：只是为了让代码更清晰。
- `int main() {`：程序入口函数。C++ 程序从 `main` 开始执行，`int` 表示函数返回一个整数。
- `std::cout << "tiny_httpd starting..." << std::endl;`：向终端输出一行文字。`std::cout` 是标准输出，`<<` 表示把内容送到输出流，`std::endl` 表示换行并刷新输出。
- `return 0;`：告诉操作系统程序正常结束。通常 `0` 表示成功。
- `}`：结束 `main` 函数。

保存退出：

```
Esc
:wq
Enter
```

创建最小 CMake

打开：

```
vim CMakeLists.txt
```

输入：

```
cmake_minimum_required(VERSION 3.16)

project(TinyHttpServer LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(tiny_httpd src/main.cpp)
```

逐行解释：

- `cmake_minimum_required(VERSION 3.16)`：要求 CMake 版本至少是 3.16。
- 空行：提高可读性。
- `project(TinyHttpServer LANGUAGES CXX)`：定义项目名为 `TinyHttpServer`，使用 C++ 语言。`CXX` 是 CMake 中 C++ 的名字。
- `set(CMAKE_CXX_STANDARD 17)`：设置 C++ 标准为 C++17。
- `set(CMAKE_CXX_STANDARD_REQUIRED ON)`：要求必须使用 C++17，如果编译器不支持就报错。

- `add_executable(tiny_httpd src/main.cpp)`：生成一个名为 `tiny_httpd` 的可执行程序，源文件是 `src/main.cpp`。

构建运行

```
cmake -S . -B build
cmake --build build
./build/tiny_httpd
```

逐行解释：

- `cmake -S . -B build`： `-S .` 表示源码目录是当前目录， `-B build` 表示构建文件放到 `build` 目录。
- `cmake --build build`：按照 `build` 目录中的构建规则编译项目。
- `./build/tiny_httpd`：运行编译出来的程序。

期望输出：

```
tiny_httpd starting...
```

对照当前项目

当前项目的 `CMakeLists.txt` 更复杂，因为它已经拆成：

- `tiny_http_core`：核心库。
- `parser_tests`：测试程序。
- `tiny_httpd`：Linux 下的服务器程序。

本节只做最小骨架，后面会逐步长成当前项目。

第 2 课：写第一个 HTTP 响应

本节目标

先不写 `socket`，只写一个函数，生成浏览器能看懂的 HTTP 响应字符串。

理论知识：HTTP 响应格式

HTTP 响应由 4 部分组成：

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 5
Connection: close

hello
```

解释：

- `HTTP/1.1 200 OK`：状态行。
- `Content-Type`：告诉客户端内容类型。
- `Content-Length`：告诉客户端响应体有多少字节。
- `Connection`：告诉客户端连接是否关闭。
- 空行：Header 结束。
- `hello`：响应体。

注意：HTTP 规范中行尾是 `\r\n`，不是普通的 `\n`。

创建文件

```
touch include/tiny_http/http_response.h src/http_response.cpp
```

编写头文件

打开：

```
vim include/tiny_http/http_response.h
```

输入：

```

#pragma once

#include <map>
#include <string>

namespace tiny_http {

class HttpResponse {
public:
    explicit HttpResponse(int status = 200, std::string reason = "OK");

    void set_header(std::string key, std::string value);
    void set_body(std::string body, std::string content_type = "text/plain; charset=utf-8");
    void set_keep_alive(bool keep_alive);
    std::string serialize() const;

private:
    int status_;
    std::string reason_;
    std::map<std::string, std::string> headers_;
    std::string body_;
    bool keep_alive_{false};
};

} // namespace tiny_http

```

逐行解释：

- `#pragma once`：防止头文件被重复包含。
- `#include <map>`：使用 `std::map` 保存 Header 键值对。
- `#include <string>`：使用 `std::string` 保存字符串。
- `namespace tiny_http {`：把项目代码放进 `tiny_http` 命名空间，避免和其他库名字冲突。
- `class HttpResponse {`：定义一个 HTTP 响应类。
- `public:`：下面是外部可以调用的接口。
- `explicit HttpResponse(...)`：构造函数。`explicit` 防止隐式类型转换。默认状态码是 200，原因短语是 OK。

- `void set_header(...)`: 设置一个响应头。
- `void set_body(...)`: 设置响应体, 同时设置内容类型。
- `void set_keep_alive(...)`: 设置是否保持连接。
- `std::string serialize() const`: 把响应对象转换为 HTTP 文本。 `const` 表示这个函数不会修改对象。
- `private:`: 下面是类内部数据, 外部不能直接访问。
- `int status_`: 保存状态码, 例如 200、404。
- `std::string reason_`: 保存原因短语, 例如 OK、Not Found。
- `std::map<...> headers_`: 保存响应头。
- `std::string body_`: 保存响应体。
- `bool keep_alive_{false}`: 默认不保持连接。
- `};`: 结束类定义。
- `} // namespace tiny_http`: 结束命名空间。

编写实现文件

打开:

```
vim src/http_response.cpp
```

输入:

```

#include "tiny_http/http_response.h"

#include <sstream>

namespace tiny_http {

HttpResponse::HttpResponse(int status, std::string reason)
    : status_(status), reason_(std::move(reason)) {}

void HttpResponse::set_header(std::string key, std::string value) {
    headers_[std::move(key)] = std::move(value);
}

void HttpResponse::set_body(std::string body, std::string content_type) {
    body_ = std::move(body);
    set_header("Content-Type", std::move(content_type));
    set_header("Content-Length", std::to_string(body_.size()));
}

void HttpResponse::set_keep_alive(bool keep_alive) {
    keep_alive_ = keep_alive;
}

std::string HttpResponse::serialize() const {
    std::ostringstream response;
    response << "HTTP/1.1 " << status_ << ' ' << reason_ << "\r\n";
    for (const auto& [key, value] : headers_) {
        response << key << ": " << value << "\r\n";
    }
    response << "Connection: " << (keep_alive_ ? "keep-alive" : "close") << "\r\n";
    response << "\r\n";
    response << body_;
    return response.str();
}

} // namespace tiny_http

```

逐行解释：

- `#include "tiny_http/http_response.h"`：引入刚才写的类声明。
- `#include <sstream>`：使用 `std::ostringstream` 拼接字符串。
- `namespace tiny_http {`：实现也放进相同命名空间。
- `HttpResponse::HttpResponse(...)`：实现构造函数。
- `: status_(status), reason_(std::move(reason))`：成员初始化列表。把传入的状态码保存到 `status_`，把 `reason` 移动到 `reason_`，减少拷贝。
- `{}`：构造函数函数体为空，因为成员已经在初始化列表里设置好了。
- `void HttpResponse::set_header(...)`：实现设置 Header 的函数。
- `headers_[std::move(key)] = std::move(value);`：把 key 和 value 存进 map。 `std::move` 表示可以转移字符串资源，减少拷贝。
- `void HttpResponse::set_body(...)`：实现设置响应体。
- `body_ = std::move(body);`：保存响应体。
- `set_header("Content-Type", ...)`：设置内容类型。
- `set_header("Content-Length", std::to_string(body_.size()))`：根据 body 长度自动设置 Content-Length。
- `void HttpResponse::set_keep_alive(...)`：保存连接是否保持。
- `std::ostringstream response;`：创建一个字符串输出流，用来拼接响应。
- `response << "HTTP/1.1 " ...`：写入状态行。
- `for (const auto& [key, value] : headers_)`：遍历所有 Header。这里用的是 C++17 结构化绑定。
- `response << key << ": " << value << "\r\n";`：写入一行 Header。
- `response << "Connection: " ...`：写入连接状态。
- `response << "\r\n";`：写入空行，表示 Header 结束。
- `response << body_;`：写入响应体。
- `return response.str();`：把输出流转换成字符串并返回。

修改 CMake

现在要让 CMake 编译 `src/http_response.cpp`。

可以先改成：


```
cmake_minimum_required(VERSION 3.16)

project(TinyHttpServer LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(tiny_httpd
    src/main.cpp
    src/http_response.cpp
)

target_include_directories(tiny_httpd PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)
```

新增内容解释：

- `src/http_response.cpp`：把响应实现文件加入编译。
- `target_include_directories(...)`：告诉编译器头文件在 `include` 目录。
- `${CMAKE_CURRENT_SOURCE_DIR}`：当前 `CMakeLists.txt` 所在目录。

修改 main 测试响应

打开：

```
vim src/main.cpp
```

改成：

```
#include "tiny_http/http_response.h"

#include <iostream>

int main() {
    tiny_http::HttpResponse response(200, "OK");
    response.set_body("hello");
    response.set_keep_alive(false);
    std::cout << response.serialize();
    return 0;
}
```

逐行解释：

- `#include "tiny_http/http_response.h"`：引入我们自己的响应类。
- `#include <iostream>`：使用 `std::cout` 输出。
- `int main() {`：程序入口。
- `tiny_http::HttpResponse response(200, "OK");`：创建一个 200 OK 响应对象。因为类在 `tiny_http` 命名空间里，所以要写 `tiny_http::`。
- `response.set_body("hello");`：设置响应体为 `hello`，同时自动设置 `Content-Type` 和 `Content-Length`。
- `response.set_keep_alive(false);`：设置连接写完后关闭。
- `std::cout << response.serialize();`：把响应对象序列化成 HTTP 文本并输出。
- `return 0;`：程序正常结束。

构建运行

```
cmake -S . -B build
cmake --build build
./build/tiny_httpd
```

你应该看到类似：

```
HTTP/1.1 200 OK
Content-Length: 5
Content-Type: text/plain; charset=utf-8
Connection: close

hello
```

对照当前项目

当前项目的 `src/http_response.cpp` 还多了一个辅助函数：

```
HttpResponse HttpResponse::text(int status, const std::string& reason, const std::string&
body)
```

它用于快速生成纯文本响应，例如 404、403、405。

第 3 课：定义 HTTP 请求对象

本节目标

把原始 HTTP 请求字符串解析成结构化对象之前，先定义这个对象长什么样。

创建文件

```
touch include/tiny_http/http_request.h
```

编写代码

打开：

```
vim include/tiny_http/http_request.h
```

输入：

```

#pragma once

#include <map>
#include <string>

namespace tiny_http {

enum class ParseStatus {
    Complete,
    Incomplete,
    BadRequest
};

struct HttpRequest {
    std::string method;
    std::string target;
    std::string version;
    std::map<std::string, std::string> headers;
    std::string body;

    bool keep_alive() const;
    std::string header(const std::string& key) const;
};

struct ParseResult {
    ParseStatus status{ParseStatus::Incomplete};
    HttpRequest request;
    std::string error;
    std::size_t consumed{0};
};

} // namespace tiny_http

```

逐行解释：

- `#pragma once`：防止头文件重复包含。
- `#include <map>`：Header 用 map 存储。
- `#include <string>`：请求方法、路径、版本、body 都用字符串。

- `namespace tiny_http {` : 项目命名空间。
- `enum class ParseStatus` : 定义解析状态。 `enum class` 比普通 `enum` 更安全, 不会随便和整数混用。
- `Complete` : 请求完整, 可以处理。
- `Incomplete` : 请求还没收完整, 需要继续读 `socket`。
- `BadRequest` : 请求格式错误, 应返回 400。
- `struct HttpRequest` : HTTP 请求结构体。
- `std::string method;` : 请求方法, 例如 `GET`。
- `std::string target;` : 请求目标, 例如 `/index.html`。
- `std::string version;` : HTTP 版本, 例如 `HTTP/1.1`。
- `std::map<std::string, std::string> headers;` : Header 键值对。
- `std::string body;` : 请求体, `GET` 通常为空, `POST` 可能有内容。
- `bool keep_alive() const;` : 判断是否长连接。
- `std::string header(...) const;` : 根据 key 查询 Header。
- `struct ParseResult` : 解析器返回结果。
- `ParseStatus status{ParseStatus::Incomplete};` : 默认状态是不完整。
- `HttpRequest request;` : 解析出来的请求对象。
- `std::string error;` : 解析失败原因。
- `std::size_t consumed{0};` : 本次完整请求消耗了多少字节。

理论知识：为什么需要 Incomplete

很多初学者会以为一次 `recv()` 就能收到完整请求, 这是错的。

TCP 是字节流。客户端发:

```
GET / HTTP/1.1\r\nHost: localhost\r\n\r\n
```

服务器可能第一次只收到:

```
GET / HTTP/1.1\r\nHo
```

第二次才收到剩下的:

```
st: localhost\r\n\r\n
```

所以解析器必须能回答: “现在数据还不够, 不要报错, 继续等。”

本节练习

1. 写出 `GET /hello.txt HTTP/1.1` 对应的 `HttpRequest` 字段。
2. 解释 `ParseStatus::Incomplete` 和 `ParseStatus::BadRequest` 的区别。
3. 思考 `consumed` 在 `Keep-Alive` 里为什么重要。

第 4 课：编写 HTTP 解析器接口

本节目标

创建 `HttpParser` 类，先定义接口，再逐步实现。

创建文件

```
touch include/tiny_http/http_parser.h src/http_parser.cpp
```

编写头文件

打开：

```
vim include/tiny_http/http_parser.h
```

输入：

```
#pragma once

#include "tiny_http/http_request.h"

#include <string>

namespace tiny_http {

class HttpParser {
public:
    ParseResult parse(const std::string& buffer) const;

private:
    static bool parse_request_line(const std::string& line, HttpRequest& request,
std::string& error);
    static bool parse_header_line(const std::string& line, HttpRequest& request,
std::string& error);
};

} // namespace tiny_http
```

逐行解释：

- `#pragma once`：防止重复包含。
- `#include "tiny_http/http_request.h"`：解析器需要使用 `HttpRequest` 和 `ParseResult`。
- `#include <string>`：解析输入是字符串。
- `namespace tiny_http {`：命名空间。
- `class HttpParser {`：定义解析器类。
- `public:`：公开接口。
- `ParseResult parse(...) const;`：输入原始请求缓冲区，输出解析结果。`const` 表示解析器本身不保存状态。
- `private:`：私有辅助函数。
- `static bool parse_request_line(...)`：解析请求行。
- `static bool parse_header_line(...)`：解析 Header 行。

理论知识：为什么 parse 是 const

当前解析器是“无状态解析器”。它不在对象内部保存中间状态，而是每次根据传入的 `buffer` 直接解析。

这样有两个好处：

- 更容易测试。
- 多线程使用时更安全。

本节练习

1. 解释 `public` 和 `private` 的区别。
2. 解释为什么 `parse_request_line` 可以是 `static`。
3. 解释 `const std::string& buffer` 为什么用引用。

提示：

- `const std::string&` 表示不拷贝大字符串，也不允许函数修改它。

第 5 课：实现字符串工具函数 trim 和 lower

本节目标

实现 Header 解析需要的小工具：

- 去掉前后空白： `trim`
- 转小写： `lower`

打开实现文件

```
vim src/http_parser.cpp
```

输入第一部分：


```

#include "tiny_http/http_parser.h"

#include <algorithm>
#include <cctype>
#include <sstream>

namespace tiny_http {

namespace {

std::string trim(std::string value) {
    auto not_space = [](unsigned char c) { return !std::isspace(c); };
    value.erase(value.begin(), std::find_if(value.begin(), value.end(), not_space));
    value.erase(std::find_if(value.rbegin(), value.rend(), not_space).base(),
value.end());
    return value;
}

std::string lower(std::string value) {
    std::transform(value.begin(), value.end(), value.begin(), [](unsigned char c) {
        return static_cast<char>(std::tolower(c));
    });
    return value;
}

} // namespace

} // namespace tiny_http

```

逐行解释：

- `#include "tiny_http/http_parser.h"`：引入解析器声明。
- `#include <algorithm>`：使用 `std::find_if` 和 `std::transform`。
- `#include <cctype>`：使用 `std::isspace` 和 `std::tolower`。
- `#include <sstream>`：后面解析请求行会用到字符串流。
- `namespace tiny_http {`：进入项目命名空间。
- `namespace {`：匿名命名空间，里面的函数只在当前 `.cpp` 文件可见，避免外部调用。

- `std::string trim(std::string value)`：定义 trim 函数。参数按值传入，因为函数会修改副本并返回。
- `auto not_space = ...`：定义一个 lambda，用来判断字符是不是非空白。
- `std::isspace(c)`：判断字符是否为空格、换行、制表符等。
- `!std::isspace(c)`：不是空白就返回 true。
- `value.erase(value.begin(), std::find_if(...))`：删除开头连续空白。
- `value.erase(std::find_if(value.rbegin(), value.rend(), not_space).base(), value.end())`：删除末尾连续空白。
- `return value;`：返回处理后的字符串。
- `std::string lower(std::string value)`：定义转小写函数。
- `std::transform(...)`：遍历字符串，把每个字符转换后写回原位置。
- `std::tolower(c)`：把字符转成小写。
- `static_cast<char>(...)`：把返回值转回 char。
- `} // namespace`：结束匿名命名空间。

理论知识：为什么要 trim 和 lower

HTTP Header 可能这样写：

```
Content-Length:    11
```

如果不 trim，拿到的值是 " 11"，转换数字时容易出问题。

Header 名大小写不敏感：

```
Host: localhost
host: localhost
HOST: localhost
```

它们应该被视为同一个字段，所以内部统一转小写。

本节练习

1. 手写 `trim(" abc ")` 的结果。
2. 手写 `lower("Content-Length")` 的结果。
3. 思考为什么匿名命名空间适合放工具函数。

第 6 课：实现 HttpRequest 的 header 和 keep_alive

本节目标

实现两个请求对象方法：

- 查询 Header。
- 判断是否长连接。

在 `src/http_parser.cpp` 继续添加

把下面代码放在匿名命名空间结束后、`tiny_http` 命名空间内部：

```
bool HttpRequest::keep_alive() const {
    const auto connection = lower(header("Connection"));
    if (version == "HTTP/1.1") {
        return connection != "close";
    }
    return connection == "keep-alive";
}

std::string HttpRequest::header(const std::string& key) const {
    const auto it = headers.find(lower(key));
    return it == headers.end() ? std::string{} : it->second;
}
```

逐行解释：

- `bool HttpRequest::keep_alive() const`：实现 `HttpRequest` 的成员函数，返回是否保持连接。
- `const auto connection = lower(header("Connection"));`：取出 `Connection` Header，并转成小写。
- `if (version == "HTTP/1.1")`：HTTP/1.1 默认长连接。
- `return connection != "close";`：HTTP/1.1 中只要没有明确写 `Connection: close`，就保持连接。
- `return connection == "keep-alive";`：HTTP/1.0 默认短连接，只有明确写 `keep-alive` 才保持。
- `std::string HttpRequest::header(...) const`：根据 key 查询 Header。
- `const auto it = headers.find(lower(key));`：把查询 key 转小写后在 map 里找。

- `return it == headers.end() ? std::string{} : it->second;` : 如果找不到返回空字符串; 找到就返回对应值。

理论知识: HTTP/1.0 和 HTTP/1.1 的连接区别

HTTP/1.0 默认:

一个请求结束后关闭 TCP 连接。

HTTP/1.1 默认:

连接可以复用, 除非明确 `Connection: close`。

长连接好处:

- 减少 TCP 建连开销。
- 多个请求可以复用一個连接。

长连接坏处:

- 服务器需要保存连接状态。
- 空闲连接要超时回收。

本节练习

1. 判断 `HTTP/1.1 + Connection: close` 是否 keep-alive。
2. 判断 `HTTP/1.0 + Connection: keep-alive` 是否 keep-alive。
3. 判断 `HTTP/1.1` 没有 Connection Header 是否 keep-alive。

第 7 课: 实现请求行解析

本节目标

解析这一行:

`GET /index.html HTTP/1.1`

在 `src/http_parser.cpp` 继续添加

```
bool HttpParser::parse_request_line(const std::string& line, HttpRequest& request,
std::string& error) {
    std::istringstream parts(line);
    if (!(parts >> request.method >> request.target >> request.version)) {
        error = "malformed request line";
        return false;
    }
    std::string extra;
    if (parts >> extra) {
        error = "too many fields in request line";
        return false;
    }
    if (request.version != "HTTP/1.0" && request.version != "HTTP/1.1") {
        error = "unsupported HTTP version";
        return false;
    }
    if (request.target.empty() || request.target[0] != '/') {
        error = "request target must start with /";
        return false;
    }
    return true;
}
```

逐行解释：

- `bool HttpParser::parse_request_line(...)`：实现解析请求行的私有函数。
- `const std::string& line`：输入的一行请求行。
- `HttpRequest& request`：输出解析结果，写入 `request`。
- `std::string& error`：如果失败，写入错误原因。
- `std::istringstream parts(line);`：把字符串变成输入流，可以用 `>>` 按空白拆字段。
- `if (!(parts >> request.method >> request.target >> request.version))`：尝试读取 3 个字段，如果失败说明格式不完整。
- `error = "malformed request line";`：记录错误原因。
- `return false;`：解析失败。
- `std::string extra;`：准备检查是否有多余字段。
- `if (parts >> extra)`：如果还能读到第 4 个字段，说明请求行字段太多。

- `error = "too many fields in request line";`：记录字段过多。
- `if (request.version != "HTTP/1.0" && request.version != "HTTP/1.1")`：只支持 HTTP/1.0 和 HTTP/1.1。
- `error = "unsupported HTTP version";`：记录版本不支持。
- `if (request.target.empty() || request.target[0] != '/')`：target 不能为空，而且必须以 `/` 开头。
- `error = "request target must start with /";`：记录 target 错误。
- `return true;`：所有检查通过，解析成功。

理论知识：为什么要严格检查

服务器的入口是外部输入，不能相信客户端一定发正确数据。严格检查请求行可以尽早拒绝坏请求，避免后续逻辑处理异常状态。

本节练习

判断下面请求行是否合法：

```
GET / HTTP/1.1
GET relative HTTP/1.1
GET / HTTP/2
GET / HTTP/1.1 abc
```

第 8 课：实现 Header 行解析

本节目标

解析 Header：

```
Host: localhost
Content-Length: 11
```

在 `src/http_parser.cpp` 继续添加

```
bool HttpParser::parse_header_line(const std::string& line, HttpRequest& request,
std::string& error) {
    const std::size_t colon = line.find(':');
    if (colon == std::string::npos) {
        error = "malformed header";
        return false;
    }
    const std::string key = lower(trim(line.substr(0, colon)));
    const std::string value = trim(line.substr(colon + 1));
    if (key.empty()) {
        error = "empty header key";
        return false;
    }
    request.headers[key] = value;
    return true;
}
```

逐行解释：

- `bool HttpParser::parse_header_line(...)`：实现 Header 行解析函数。
- `const std::size_t colon = line.find(':');`：查找冒号位置。
- `if (colon == std::string::npos)`：如果找不到冒号，说明 Header 格式错误。
- `error = "malformed header";`：记录错误。
- `return false;`：解析失败。
- `line.substr(0, colon)`：取冒号左边作为 key。
- `trim(...)`：去掉 key 两边空白。
- `lower(...)`：把 key 转小写。
- `line.substr(colon + 1)`：取冒号右边作为 value。
- `trim(...)`：去掉 value 两边空白。
- `if (key.empty())`：key 不能为空。
- `request.headers[key] = value;`：保存 Header。
- `return true;`：解析成功。

理论知识：Header 为什么大小写不敏感

HTTP 协议规定 Header field name 大小写不敏感。也就是说：

```
Host  
host  
HOST
```

都表示同一个 Header。项目内部统一小写，可以让后续查询更简单。

本节练习

1. 解析 `Content-Length: 11` 后 value 是什么？
2. 解析 `BadHeader` 会发生什么？
3. 解析 `: abc` 会发生什么？

第 9 课：实现完整 parse 函数

本节目标

把请求行、Header、Body 串起来，完成 HTTP 请求解析。

在 `src/http_parser.cpp` 继续添加

```

ParseResult HttpParser::parse(const std::string& buffer) const {
    ParseResult result;
    const std::size_t header_end = buffer.find("\r\n\r\n");
    if (header_end == std::string::npos) {
        if (buffer.size() > 8192) {
            result.status = ParseStatus::BadRequest;
            result.error = "request header is too large";
        }
        return result;
    }

    const std::string header_block = buffer.substr(0, header_end);
    std::istringstream stream(header_block);
    std::string line;

    if (!std::getline(stream, line)) {
        result.status = ParseStatus::BadRequest;
        result.error = "missing request line";
        return result;
    }
    if (!line.empty() && line.back() == '\r') {
        line.pop_back();
    }
    if (!parse_request_line(line, result.request, result.error)) {
        result.status = ParseStatus::BadRequest;
        return result;
    }

    while (std::getline(stream, line)) {
        if (!line.empty() && line.back() == '\r') {
            line.pop_back();
        }
        if (line.empty()) {
            break;
        }
        if (!parse_header_line(line, result.request, result.error)) {

```

```

        result.status = ParseStatus::BadRequest;
        return result;
    }
}

std::size_t body_start = header_end + 4;
std::size_t content_length = 0;
const std::string content_length_text = result.request.header("Content-Length");
if (!content_length_text.empty()) {
    try {
        content_length = static_cast<std::size_t>(std::stoul(content_length_text));
    } catch (...) {
        result.status = ParseStatus::BadRequest;
        result.error = "invalid Content-Length";
        return result;
    }
}

if (buffer.size() < body_start + content_length) {
    result.status = ParseStatus::Incomplete;
    return result;
}

result.request.body = buffer.substr(body_start, content_length);
result.consumed = body_start + content_length;
result.status = ParseStatus::Complete;
return result;
}

```

逐段解释：

第一段：

- `ParseResult result;`：创建默认解析结果，默认是 `Incomplete`。
- `buffer.find("\r\n\r\n")`：查找 Header 结束位置。
- 找不到时，如果缓冲区超过 8192 字节，认为 Header 太大，返回坏请求。
- 找不到且没超限，说明数据还没收齐，返回默认 `Incomplete`。

第二段：

- `header_block`：取出 Header 部分，不包含 Body。
- `std::istringstream stream(header_block)`：把 Header 部分变成逐行读取的流。
- `std::getline(stream, line)`：读取第一行，也就是请求行。

第三段：

- 如果第一行缺失，返回 `BadRequest`。
- 如果行尾有 `\r`，删掉它，因为 `getline` 默认按 `\n` 分割，会留下 `\r`。
- 调用 `parse_request_line` 解析请求行。

第四段：

- 循环读取 Header 行。
- 每行同样去掉末尾 `\r`。
- 空行表示 Header 结束。
- 调用 `parse_header_line` 解析每个 Header。

第五段：

- `body_start = header_end + 4`：`\r\n\r\n` 占 4 个字符，所以 Body 从后面开始。
- 默认 `content_length = 0`。
- 从 Header 中读取 `Content-Length`。
- 如果存在，尝试转换为数字。
- 转换失败则返回 `BadRequest`。

第六段：

- 如果缓冲区长度小于 `body_start + content_length`，说明 Body 没收齐，返回 `Incomplete`。
- 否则取出 Body。
- 设置 `consumed`。
- 设置状态为 `Complete`。
- 返回结果。

理论知识：为什么 HTTP 解析不是简单 `sscanf`

简单 `sscanf` 只能处理非常理想的请求行，不能处理：

- Header 大小写。
- Header 前后空格。
- Body 是否收齐。
- 不完整请求。
- 多个请求粘在一起。

所以项目用分阶段解析，而不是一行 `sscanf` 解决。

本节练习

1. 构造一个没有 `\r\n\r\n` 的请求，看看结果是什么。
2. 构造一个 `Content-Length: abc` 的请求，看看结果是什么。
3. 构造一个 POST 请求，Body 长度刚好等于 Content-Length。

第 10 课：写单元测试验证解析器

本节目标

写测试，而不是靠肉眼猜代码对不对。

创建测试文件

```
touch tests/parser_tests.cpp
```

打开：

```
vim tests/parser_tests.cpp
```

输入：

```

#include "tiny_http/http_parser.h"
#include "tiny_http/http_response.h"

#include <cassert>
#include <iostream>

using namespace tiny_http;

int main() {
    HttpParser parser;

    const std::string raw =
        "GET /index.html HTTP/1.1\r\n"
        "Host: localhost\r\n"
        "Connection: keep-alive\r\n"
        "\r\n";

    const auto result = parser.parse(raw);
    assert(result.status == ParseStatus::Complete);
    assert(result.request.method == "GET");
    assert(result.request.target == "/index.html");
    assert(result.request.header("host") == "localhost");
    assert(result.request.keep_alive());

    HttpResponse response(200, "OK");
    response.set_body("hello");
    const std::string text = response.serialize();
    assert(text.find("HTTP/1.1 200 OK\r\n") == 0);

    std::cout << "parser tests passed\n";
    return 0;
}

```

逐行解释：

- `#include "tiny_http/http_parser.h"`：引入解析器。
- `#include "tiny_http/http_response.h"`：引入响应类。
- `#include <cassert>`：使用 `assert` 断言。

- `#include <iostream>` : 输出测试通过信息。
- `using namespace tiny_http;` : 后面不用每次写 `tiny_http::`。
- `HttpParser parser;` : 创建解析器。
- `const std::string raw = ...` : 构造一个原始 HTTP 请求字符串。
- 每个字符串字面量相邻时, C++ 会自动拼接。
- `const auto result = parser.parse(raw);` : 解析请求。
- `assert(...)` : 如果条件为 `false`, 程序会中止, 说明测试失败。
- `result.status == ParseStatus::Complete` : 要求解析结果完整。
- `result.request.method == "GET"` : 检查 `method`。
- `result.request.target == "/index.html"` : 检查 `target`。
- `result.request.header("host") == "localhost"` : 检查 `Header` 查询。
- `result.request.keep_alive()` : 检查长连接判断。
- `HttpResponse response(200, "OK");` : 创建响应。
- `response.set_body("hello");` : 设置 `body`。
- `response.serialize()` : 生成响应文本。
- `text.find(...) == 0` : 确认响应以状态行开头。
- `std::cout << "parser tests passed\n";` : 输出测试通过。
- `return 0;` : 测试程序正常结束。

修改 CMake

逐步项目可以这样写:

```
add_library(tiny_http_core
    src/http_parser.cpp
    src/http_response.cpp
)

target_include_directories(tiny_http_core PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

add_executable(parser_tests tests/parser_tests.cpp)
target_link_libraries(parser_tests PRIVATE tiny_http_core)
```

解释:

- `add_library(tiny_http_core ...)` : 把通用代码编译成库。

- `PUBLIC`：使用这个库的人也能看到 `include` 路径。
- `add_executable(parser_tests ...)`：生成测试程序。
- `target_link_libraries(...)`：测试程序链接核心库。

运行测试

```
cmake -S . -B build
cmake --build build
./build/parser_tests
```

如果使用 CTest:

```
ctest --test-dir build --output-on-failure
```

解释:

- `ctest`：CMake 配套测试工具。
- `--test-dir build`：测试信息在 `build` 目录里。
- `--output-on-failure`：失败时显示输出，方便排错。

本节练习

增加 3 个测试:

1. 不完整请求应该返回 `Incomplete`。
2. `GET relative HTTP/1.1` 应返回 `BadRequest`。
3. `POST` 带 `Body` 应正确解析 `Body`。

第 11 课：添加 MIME 类型识别

本节目标

让服务器返回文件时知道文件类型。

创建文件

```
touch include/tiny_http/mime_types.h src/mime_types.cpp
```

编写头文件

```
#pragma once

#include <string>

namespace tiny_http {

std::string mime_type_for_path(const std::string& path);

} // namespace tiny_http
```

逐行解释：

- `#pragma once`：防止重复包含。
- `#include <string>`：函数参数和返回值都用 `string`。
- `namespace tiny_http`：项目命名空间。
- `std::string mime_type_for_path(...)`：根据文件路径返回 MIME 类型。

编写实现

```
#include "tiny_http/mime_types.h"

#include <algorithm>
#include <cctype>
#include <unordered_map>

namespace tiny_http {

std::string mime_type_for_path(const std::string& path) {
    const auto dot = path.find_last_of('.');
    if (dot == std::string::npos) {
        return "application/octet-stream";
    }

    std::string ext = path.substr(dot);
    std::transform(ext.begin(), ext.end(), ext.begin(), [](unsigned char c) {
        return static_cast<char>(std::tolower(c));
    });

    static const std::unordered_map<std::string, std::string> types{
        {".html", "text/html; charset=utf-8"},
        {".css", "text/css; charset=utf-8"},
        {".js", "application/javascript; charset=utf-8"},
        {".txt", "text/plain; charset=utf-8"},
        {".png", "image/png"}
    };

    const auto it = types.find(ext);
    return it == types.end() ? "application/octet-stream" : it->second;
}

} // namespace tiny_http
```

逐行解释：

- `#include "tiny_http/mime_types.h"`：引入函数声明。
- `#include <algorithm>`：使用 `std::transform`。

- `#include <cctype>`：使用 `std::tolower`。
- `#include <unordered_map>`：使用哈希表保存扩展名映射。
- `path.find_last_of('.')`：查找最后一个点，获取扩展名位置。
- `if (dot == std::string::npos)`：如果没有点，说明没有扩展名。
- `return "application/octet-stream";`：未知二进制流类型。
- `std::string ext = path.substr(dot);`：从点开始截取扩展名，例如 `.html`。
- `std::transform(...)`：把扩展名转小写。
- `static const std::unordered_map...`：定义扩展名到 MIME 的映射表。`static` 表示只初始化一次。
- `types.find(ext)`：查找扩展名。
- `return it == types.end() ? ...`：找不到返回默认类型，找到返回映射值。

理论知识：MIME 是谁用的

浏览器收到响应后，会看：

```
Content-Type: text/html; charset=utf-8
```

然后决定：

- 当 HTML 渲染。
- 当 CSS 应用样式。
- 当图片显示。
- 当下载文件。

本节练习

1. 添加 `.json -> application/json; charset=utf-8`。
2. 添加 `.jpg -> image/jpeg`。
3. 写一个测试验证 `.html`。

第 12 课：准备静态网页目录

本节目标

创建 `www/index.html`，让服务器未来能返回真实页面。

创建文件

```
touch www/index.html
```

打开:

```
vim www/index.html
```

输入:

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8">
  <title>Tiny HTTP Server</title>
</head>
<body>
  <h1>Tiny HTTP Server</h1>
  <p>hello from tiny_httpd</p>
</body>
</html>
```

逐行解释:

- `<!doctype html>`: 告诉浏览器这是 HTML5 文档。
- `<html lang="zh-CN">`: HTML 根元素, 语言是简体中文。
- `<head>`: 放页面元信息。
- `<meta charset="utf-8">`: 设置字符编码为 UTF-8, 避免中文乱码。
- `<title>...</title>`: 浏览器标签页标题。
- `</head>`: 结束 head。
- `<body>`: 页面可见内容。
- `<h1>...</h1>`: 一级标题。
- `<p>...</p>`: 段落。
- `</body>`: 结束 body。
- `</html>`: 结束 HTML 文档。

查看文件:

```
cat www/index.html
```

理论知识：静态文件服务

静态文件服务的意思是：

```
浏览器请求 /index.html
服务器读取 www/index.html
服务器把文件内容作为 HTTP 响应体返回
```

后面写服务器时，`www` 会作为 document root，也就是站点根目录。

第 13 课：写一个最小阻塞版 Linux Socket 服务器

本节目标

先用最简单方式理解 socket。暂时不用 epoll、线程池。

理论知识：TCP 服务器六步

```
socket    创建 socket
bind      绑定 IP 和端口
listen    开始监听
accept    接受连接
recv      接收请求
send      发送响应
close     关闭连接
```

监听 fd 和客户端 fd 不一样：

- `listen_fd`：只负责接受新连接。
- `client_fd`：负责和某个客户端收发数据。

示例代码

这段代码是教学用的阻塞版，不是最终项目代码：

```

int server_fd = socket(AF_INET, SOCK_STREAM, 0);
sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = INADDR_ANY;
addr.sin_port = htons(8080);
bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
listen(server_fd, SOMAXCONN);
int client_fd = accept(server_fd, nullptr, nullptr);
char buffer[4096];
recv(client_fd, buffer, sizeof(buffer), 0);
std::string response = "HTTP/1.1 200 OK\r\nContent-Length: 5\r\nConnection:
close\r\n\r\nhello";
send(client_fd, response.data(), response.size(), 0);
close(client_fd);
close(server_fd);

```

逐行解释：

- `socket(AF_INET, SOCK_STREAM, 0)`：创建 IPv4 TCP socket。
- `sockaddr_in addr{};`：创建 IPv4 地址结构，并用 `{}` 初始化为 0。
- `addr.sin_family = AF_INET;`：地址族是 IPv4。
- `addr.sin_addr.s_addr = INADDR_ANY;`：监听所有网卡地址。
- `addr.sin_port = htons(8080);`：监听 8080 端口。`htons` 把主机字节序转成网络字节序。
- `bind(...)`：把 socket 绑定到 IP 和端口。
- `listen(...)`：开始监听连接。
- `accept(...)`：阻塞等待一个客户端连接。
- `char buffer[4096];`：创建接收缓冲区。
- `recv(...)`：读取客户端发来的请求。
- `std::string response = ...`：构造固定 HTTP 响应。
- `send(...)`：发送响应。
- `close(client_fd)`：关闭客户端连接。
- `close(server_fd)`：关闭监听 socket。

为什么这不是最终版本

阻塞版容易理解，但有严重问题：

如果一个客户端连接后不发数据，服务器会卡在 `recv`。
卡住后，其他客户端无法及时被处理。

所以最终项目使用非阻塞 `socket + epoll`。

本节练习

1. 解释 `bind` 和 `listen` 的区别。
2. 解释 `accept` 返回的是什么。
3. 解释为什么阻塞版不适合高并发。

第 14 课：封装 ServerConfig

本节目标

把端口、静态文件目录、线程数等配置集中到一个结构体。

当前项目文件：

- `include/tiny_http/config.h`

代码

```
#pragma once

#include <cstddef>
#include <string>

namespace tiny_http {

struct ServerConfig {
    int port{8080};
    std::string document_root{"www"};
    std::size_t worker_threads{4};
    int keep_alive_seconds{15};
    int max_events{1024};
    std::string log_path{"tiny_httpd.log"};
};

} // namespace tiny_http
```

逐行解释：

- `#pragma once`：防止重复包含。
- `#include <cstdint>`：使用 `std::size_t`。
- `#include <string>`：使用 `std::string`。
- `struct ServerConfig`：定义服务器配置结构体。
- `int port{8080};`：默认监听 8080 端口。
- `document_root{"www"}`：默认静态文件目录是 `www`。
- `worker_threads{4}`：默认 4 个工作线程。
- `keep_alive_seconds{15}`：Keep-Alive 空闲超时时间 15 秒。
- `max_events{1024}`：epoll 一次最多处理 1024 个事件。
- `log_path{"tiny_httpd.log"}`：日志文件路径。

理论知识：为什么需要配置结构体

如果代码里到处写 `8080`、`www`、`4`，后续修改很麻烦，也不清楚这些数字是什么意思。集中到 `ServerConfig` 后：

- 默认值清晰。
- 构造服务器时只传一个对象。
- 后续可以从配置文件读取。

本节练习

1. 把默认端口改成 9090。
2. 思考 `worker_threads` 默认应该等于 CPU 核心数吗？
3. 思考 `log_path` 为什么不写死在 `logger` 里。

第 15 课：从 0 设计 LinuxHttpServer 类

本节目标

把 socket、epoll、连接表、线程池、日志器组合成服务器类。

当前项目文件：

- `include/tiny_http/linux_server.h`

核心代码片段

```
class LinuxHttpServer {
public:
    explicit LinuxHttpServer(ServerConfig config);
    ~LinuxHttpServer();

    bool start();
    void stop();

private:
    struct Connection {
        int fd{-1};
        std::string in;
        std::string out;
        bool close_after_write{true};
        bool processing{false};
        long long last_active_ms{0};
    };
};
```

逐行解释：

- `class LinuxHttpServer`：定义 Linux HTTP 服务器类。
- `public:`：外部可以调用的接口。
- `explicit LinuxHttpServer(ServerConfig config);`：构造服务器，需要传入配置。
- `~LinuxHttpServer();`：析构函数，用于释放资源。
- `bool start();`：启动服务器。
- `void stop();`：停止服务器。
- `private:`：内部实现细节。
- `struct Connection`：每个客户端连接的状态。
- `int fd{-1};`：客户端 socket fd。
- `std::string in;`：输入缓冲区，保存收到但未处理完的数据。
- `std::string out;`：输出缓冲区，保存还没发送完的响应。
- `bool close_after_write{true};`：写完响应后是否关闭连接。
- `bool processing{false};`：是否已有工作线程正在处理这个连接的请求。
- `long long last_active_ms{0};`：最后活跃时间，用于超时关闭。

理论知识：为什么 fd 不够

fd 只是一个整数，不能保存业务状态。服务器需要知道：

- 这个连接已经收了哪些数据。
- 响应是否发完。
- 是否要 Keep-Alive。
- 是否正在处理。
- 是否超时。

这些都要放在 `Connection` 里。

本节练习

1. 解释 `in` 和 `out` 的区别。
2. 解释 `processing` 的作用。
3. 思考如果没有 `last_active_ms`，长连接会有什么问题。

第 16 课：实现监听 socket

本节目标

实现 `setup_listener()`：创建、绑定、监听 socket。

当前项目位置：

- [src/linux_server.cpp](#)

核心代码

```
listen_fd_ = ::socket(AF_INET, SOCK_STREAM, 0);
int opt = 1;
::setsockopt(listen_fd_, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
set_nonblocking(listen_fd_);

sockaddr_in address{};
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(static_cast<uint16_t>(config_.port));

::bind(listen_fd_, reinterpret_cast<sockaddr*>(&address), sizeof(address));
::listen(listen_fd_, SOMAXCONN);
```

逐行解释：

- `listen_fd_ = ::socket(AF_INET, SOCK_STREAM, 0);`：创建 TCP socket，并保存到成员变量 `listen_fd_`。
- `::`：表示调用全局命名空间里的系统函数，避免和类成员重名。
- `int opt = 1;`：准备一个开关值。
- `setsockopt(... SO_REUSEADDR ...)`：允许地址复用，避免服务器刚退出后马上重启时报端口占用。
- `set_nonblocking(listen_fd_)`：把监听 socket 设置为非阻塞。
- `sockaddr_in address{};`：创建 IPv4 地址结构。
- `address.sin_family = AF_INET;`：使用 IPv4。
- `address.sin_addr.s_addr = INADDR_ANY;`：监听所有网卡。
- `address.sin_port = htons(...)`：设置端口，转换成网络字节序。
- `bind(...)`：绑定地址。
- `listen(...)`：开始监听。

理论知识：字节序是什么

不同 CPU 存储整数的字节顺序可能不同。网络协议规定使用网络字节序，也就是大端序。端口号是整数，所以绑定前要用 `htons` 转换。

本节练习

1. 解释 `SO_REUSEADDR` 的作用。
2. 解释 `INADDR_ANY`。
3. 解释为什么端口要 `htons`。

第 17 课：实现非阻塞

本节目标

把 fd 设置为非阻塞，为 `epoll` 做准备。

核心代码

```
bool LinuxHttpServer::set_nonblocking(int fd) {
    const int flags = ::fcntl(fd, F_GETFL, 0);
    if (flags < 0) {
        return false;
    }
    return ::fcntl(fd, F_SETFL, flags | O_NONBLOCK) == 0;
}
```

逐行解释：

- `bool LinuxHttpServer::set_nonblocking(int fd)`：定义静态辅助函数，成功返回 `true`，失败返回 `false`。
- `const int flags = ::fcntl(fd, F_GETFL, 0);`：获取 `fd` 当前状态标志。
- `if (flags < 0)`：获取失败。
- `return false;`：失败返回。
- `::fcntl(fd, F_SETFL, flags | O_NONBLOCK)`：设置新的状态标志，在原有 `flags` 基础上加上 `O_NONBLOCK`。
- `== 0`：`fcntl` 成功返回 0，所以表达式为 `true`。

理论知识：非阻塞读写

非阻塞 `fd` 调用 `recv`：

```
读到数据 -> 返回正数
对端关闭 -> 返回 0
暂时没数据 -> 返回 -1, errno 为 EAGAIN 或 EWOULDBLOCK
真正错误 -> 返回 -1, errno 为其他值
```

`EAGAIN` 不是错误，只是告诉你“现在读完了”。

本节练习

1. 解释 `F_GETFL`。
2. 解释 `F_SETFL`。
3. 解释 `flags | O_NONBLOCK` 为什么用按位或。

第 18 课：创建 epoll 实例

本节目标

让内核帮我们监听很多 fd 的事件。

核心代码

```
epoll_fd_ = ::epoll_create1(Epoll_CLOEXEC);

epoll_event event{};
event.events = EPOLLIN | EPOLLET;
event.data.fd = listen_fd_;

::epoll_ctl(epoll_fd_, EPOLL_CTL_ADD, listen_fd_, &event);
```

逐行解释：

- `epoll_fd_ = ::epoll_create1(Epoll_CLOEXEC);`：创建 epoll 实例，返回 epoll fd。
- `EPOLL_CLOEXEC`：进程执行新程序时自动关闭这个 fd，属于资源安全设置。
- `epoll_event event{};`：创建事件结构并初始化为 0。
- `event.events = EPOLLIN | EPOLLET;`：关注可读事件，并使用边缘触发模式。
- `event.data.fd = listen_fd_;`：事件发生时让 epoll 返回这个 fd。
- `epoll_ctl(...)`：把监听 fd 添加到 epoll。
- `EPOLL_CTL_ADD`：添加操作。

理论知识：epoll 不负责读写

epoll 只告诉你：

某个 fd 可能可读或可写。

真正读写仍然要自己调用：

```
recv
send
accept
```

本节练习

1. 解释 `EPOLLIN`。
2. 解释 `EPOLLET`。
3. 解释 `event.data.fd` 的作用。

第 19 课：编写事件循环

本节目标

实现服务器主循环：等待事件，然后分发处理。

核心代码结构

```
while (running_) {
    const int count = ::epoll_wait(epoll_fd_, events.data(), config_.max_events, 100);
    for (int i = 0; i < count; ++i) {
        const int fd = events[i].data.fd;
        const uint32_t flags = events[i].events;

        if (fd == listen_fd_) {
            accept_clients();
            continue;
        }
        if ((flags & EPOLLIN) != 0) {
            handle_read(fd);
        }
        if ((flags & EPOLLOUT) != 0) {
            handle_write(fd);
        }
    }
    sweep_idle_connections();
}
```

逐行解释：

- `while (running_)`：只要服务器还在运行，就不断循环。
- `epoll_wait(...)`：等待事件，最多等待 100 毫秒。
- `events.data()`：事件数组地址。

- `config_.max_events`：最多接收多少个事件。
- `100`：超时时间，单位毫秒。
- `for (int i = 0; i < count; ++i)`：遍历本轮返回的事件。
- `fd = events[i].data.fd`：取出发生事件的 fd。
- `flags = events[i].events`：取出事件类型。
- `if (fd == listen_fd_)`：如果是监听 fd，说明有新连接。
- `accept_clients()`：接受新连接。
- `continue`：处理完新连接，进入下一个事件。
- `if ((flags & EPOLLIN) != 0)`：如果有可读事件。
- `handle_read(fd)`：读取客户端请求。
- `if ((flags & EPOLLOUT) != 0)`：如果有可写事件。
- `handle_write(fd)`：写回响应。
- `sweep_idle_connections()`：清理超时空闲连接。

理论知识：为什么 `epoll_wait` 超时是 100

如果超时是 `-1`，没有事件时会一直等，空闲连接扫描就无法定期执行。设置 100ms 后，即使没有事件，也会定期醒来检查超时连接。

本节练习

1. 解释 `flags & EPOLLIN`。
2. 解释为什么 `listen fd` 要特殊处理。
3. 解释为什么事件循环里不应该做耗时任务。

第 20 课：接受新连接

本节目标

实现 `accept_clients()`，把新客户端加入 `epoll` 和连接表。

核心代码

```
while (true) {
    sockaddr_in client_addr{};
    socklen_t len = sizeof(client_addr);
    const int client_fd = ::accept4(
        listen_fd_,
        reinterpret_cast<sockaddr*>(&client_addr),
        &len,
        SOCK_NONBLOCK | SOCK_CLOEXEC);

    if (client_fd < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            break;
        }
        break;
    }

    epoll_event event{};
    event.events = EPOLLIN | EPOLLRDHUP | EPOLLET;
    event.data.fd = client_fd;
    ::epoll_ctl(epoll_fd_, EPOLL_CTL_ADD, client_fd, &event);

    connections_.emplace(client_fd, Connection{client_fd, {}, {}, false, false,
now_ms()});
}
```

逐行解释：

- `while (true)`：ET 模式下要循环 `accept`，直到没有新连接。
- `sockaddr_in client_addr{};`：保存客户端地址信息。
- `socklen_t len = sizeof(client_addr);`：地址结构长度。
- `accept4(...)`：接受新连接。
- `listen_fd_`：监听 socket。
- `reinterpret_cast<sockaddr*>(&client_addr)`：把 IPv4 地址结构转换成通用 socket 地址指针。
- `&len`：传入地址长度。
- `SOCK_NONBLOCK | SOCK_CLOEXEC`：新 client fd 直接非阻塞，并设置 close-on-exec。

- `if (client_fd < 0)`：接受失败。
- `errno == EAGAIN || errno == EWOULDBLOCK`：当前没有更多连接了，不是真错误。
- `break`：结束 `accept` 循环。
- `epoll_event event{};`：准备注册 `client fd`。
- `event.events = EPOLLIN | EPOLLRDHUP | EPOLLET;`：关注可读、对端关闭、边缘触发。
- `event.data.fd = client_fd;`：事件返回时带上 `client fd`。
- `epoll_ctl(... ADD ...)`：加入 `epoll`。
- `connections_.emplace(...)`：在连接表中创建连接状态。
- `now_ms()`：记录当前时间，用于超时回收。

理论知识：为什么 `accept` 也要循环到 `EAGAIN`

监听 `fd` 使用 `ET` 模式。一次通知可能对应多个等待中的连接。如果只 `accept` 一个，其他连接可能留在队列里却不再触发事件。

本节练习

1. 解释 `accept4` 和 `accept` 的区别。
2. 解释 `EPOLLRDHUP`。
3. 解释为什么要把 `client fd` 放进 `connections_`。

第 21 课：读取客户端请求

本节目标

实现读事件处理：循环 `recv`、追加输入缓冲区、调用解析器。

核心代码片段

```
char buffer[4096];
while (true) {
    const ssize_t n = ::recv(fd, buffer, sizeof(buffer), 0);
    if (n > 0) {
        connection.in.append(buffer, static_cast<std::size_t>(n));
        const auto parsed = parser_.parse(connection.in);
    } else if (n == 0) {
        close_connection(fd);
        break;
    } else {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            break;
        }
        close_connection(fd);
        break;
    }
}
```

逐行解释：

- `char buffer[4096];`：创建临时接收缓冲区。
- `while (true)`：ET 模式下循环读取。
- `recv(fd, buffer, sizeof(buffer), 0)`：从 socket 读取数据。
- `ssize_t n`：保存读取结果，可能是正数、0、-1。
- `if (n > 0)`：读到了数据。
- `connection.in.append(...)`：把读到的数据追加到连接输入缓冲区。
- `parser_.parse(connection.in)`：尝试解析完整 HTTP 请求。
- `else if (n == 0)`：对端关闭连接。
- `close_connection(fd)`：关闭本地连接。
- `else`：读取出错或暂时没数据。
- `if (errno == EAGAIN || errno == EWOULDBLOCK)`：当前已经读完，不是真错误。
- `break`：退出读循环，等待下次事件。
- 其他错误则关闭连接。

当前项目更完整的逻辑

当前项目在请求完整时：

```
raw = it->second.in.substr(0, parsed.consumed);
it->second.in.erase(0, parsed.consumed);
it->second.processing = true;
```

解释：

- `substr(0, parsed.consumed)`：取出一个完整请求。
- `erase(0, parsed.consumed)`：从输入缓冲区删除已处理部分。
- `processing = true`：标记这个连接已有任务在处理，避免重复提交。

然后提交线程池：

```
pool_.submit([this, fd, raw = std::move(raw)] {
    process_request(fd, raw);
});
```

解释：

- `pool_.submit(...)`：提交任务。
- `[this, fd, raw = std::move(raw)]`：lambda 捕获当前对象、fd 和请求字符串。
- `process_request(fd, raw)`：工作线程执行请求处理。

本节练习

1. 解释 `n > 0`、`n == 0`、`EAGAIN` 的区别。
2. 解释为什么需要 `connection.in`。
3. 解释为什么请求完整后要从 `in` 中 `erase`。

第 22 课：处理请求并生成响应

本节目标

从 `HttpRequest` 生成 `HttpResponse`，实现静态文件服务。

核心流程

1. 解析 raw 请求
2. 检查 method
3. target 映射到文件路径
4. 打开文件
5. 读取文件内容
6. 设置 MIME
7. 生成 HTTP 响应字符串

关键代码片段

```
if (request.method != "GET" && request.method != "HEAD") {  
    HttpResponse response = HttpResponse::text(405, "Method Not Allowed", "Method Not  
Allowed");  
    response.set_header("Allow", "GET, HEAD");  
    response.set_keep_alive(!close_after_write);  
    return response.serialize();  
}
```

逐行解释：

- `if (request.method != "GET" && request.method != "HEAD")`：如果方法既不是 GET 也不是 HEAD。
- `HttpResponse::text(...)`：创建纯文本错误响应。
- `405`：状态码，表示方法不允许。
- `"Method Not Allowed"`：状态说明和响应体。
- `response.set_header("Allow", "GET, HEAD");`：告诉客户端允许的方法。
- `response.set_keep_alive(!close_after_write);`：根据连接策略设置 Header。
- `return response.serialize();`：返回响应字符串。

读取文件片段：

```
std::ifstream file(path, std::ios::binary);  
if (!file) {  
    HttpResponse response = HttpResponse::text(404, "Not Found", "Not Found");  
    response.set_keep_alive(!close_after_write);  
    return response.serialize();  
}
```

逐行解释：

- `std::ifstream file(...)`：打开文件输入流。
- `std::ios::binary`：按二进制模式读取，避免不同平台换行转换。
- `if (!file)`：文件打开失败。
- 返回 404。

生成成功响应：

```
std::ostringstream contents;
contents << file.rdbuf();
HttpResponse response(200, "OK");
response.set_body(request.method == "HEAD" ? std::string{} : contents.str(),
mime_type_for_path(path));
response.set_keep_alive(!close_after_write);
return response.serialize();
```

逐行解释：

- `std::ostringstream contents;`：创建字符串输出流。
- `contents << file.rdbuf();`：把整个文件内容读入字符串流。
- `HttpResponse response(200, "OK");`：创建成功响应。
- `request.method == "HEAD" ? std::string{} : contents.str()`：如果是 HEAD，不返回 Body；否则返回文件内容。
- `mime_type_for_path(path)`：根据文件扩展名设置 Content-Type。
- `set_keep_alive(...)`：设置连接策略。
- `serialize()`：生成响应文本。

理论知识：GET 和 HEAD 的区别

- GET：返回 Header + Body。
- HEAD：只返回 Header，不返回 Body。

HEAD 常用于客户端只想知道资源是否存在、长度是多少，但不想下载内容。

本节练习

1. 解释 405 和 404 的区别。
2. 解释为什么 HEAD 不返回 Body。
3. 解释为什么读取文件用 binary 模式。

第 23 课：路径映射和路径穿越防护

本节目标

把 URL target 转成安全的本地文件路径。

核心代码

```
std::string clean = strip_query(target);
if (clean.empty() || clean == "/") {
    clean = "/index.html";
}
if (contains_path_traversal(clean)) {
    return {};
}
while (!clean.empty() && clean.front() == '/') {
    clean.erase(clean.begin());
}
return config_.document_root + "/" + clean;
```

逐行解释：

- `strip_query(target)`：去掉查询参数。例如 `/hello.txt?x=1` 变成 `/hello.txt`。
- `if (clean.empty() || clean == "/")`：如果请求根路径。
- `clean = "/index.html";`：默认返回首页。
- `contains_path_traversal(clean)`：检查是否包含危险路径。
- `return {};`：返回空字符串，表示路径非法。
- `while (...)`：去掉开头的 `/`。
- `clean.erase(clean.begin())`：删除第一个字符。
- `return config_.document_root + "/" + clean;`：拼接成 `www/index.html` 这样的路径。

理论知识：路径穿越

危险请求：

```
../../etc/passwd
```

如果服务器直接拼接：

```
www/../../etc/passwd
```

可能访问到站点根目录外的系统文件。所以当前项目拒绝包含 `..` 和反斜杠的路径。

本节练习

手算这些 target 的映射：

```
/
/index.html
/hello.txt?name=a
/../../README.md
```

第 24 课：写回响应

本节目标

处理非阻塞 `send`，支持响应分多次发完。

核心逻辑

```
const ssize_t n = ::send(fd, chunk.data(), chunk.size(), MSG_NOSIGNAL);
if (n > 0) {
    connection.out.erase(0, static_cast<std::size_t>(n));
} else if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    return;
} else {
    close_connection(fd);
    return;
}
```

逐行解释：

- `send(...)`：向客户端发送数据。
- `chunk.data()`：发送缓冲区地址。
- `chunk.size()`：希望发送的字节数。
- `MSG_NOSIGNAL`：避免对端关闭时触发 SIGPIPE 导致进程退出。
- `if (n > 0)`：成功发送了 `n` 个字节。
- `connection.out.erase(0, n)`：删除已经发送的部分。

- `else if ... EAGAIN`：暂时不能继续写，等待下一次 `EPOLLOUT`。
- `return`：保留剩余数据。
- 其他情况说明连接异常，关闭连接。

理论知识：send 不保证一次发完

`send` 返回值可能小于你传入的长度。比如你想发 10000 字节，它可能只发了 3000 字节。剩下 7000 字节必须保存在 `out` 缓冲区，等下次继续发。

本节练习

1. 解释为什么不能 `send` 一次后直接清空 `out`。
2. 解释 `MSG_NOSIGNAL`。
3. 解释写完后为什么有时关闭，有时切回 `EPOLLIN`。

第 25 课：线程池和任务队列

本节目标

理解当前项目如何让主线程负责 I/O，工作线程负责处理请求。

当前项目文件：

- `include/tiny_http/blocking_queue.h`
- `include/tiny_http/thread_pool.h`
- `src/thread_pool.cpp`

BlockingQueue 核心代码

```
void push(T value) {
    {
        std::lock_guard<std::mutex> lock(mutex_);
        if (closed_) {
            return;
        }
        queue_.push(std::move(value));
    }
    cv_.notify_one();
}
```

逐行解释：

- `void push(T value)`：向队列添加任务。
- `{ ... }`：额外作用域，让锁尽早释放。
- `std::lock_guard<std::mutex> lock(mutex_)`：加锁，保护共享队列。
- `if (closed_) return;`：如果队列已关闭，不再接受任务。
- `queue_.push(std::move(value));`：把任务移动进队列。
- `}`：离开作用域，自动解锁。
- `cv_.notify_one();`：唤醒一个等待任务的工作线程。

pop 核心：

```
std::optional<T> pop() {
    std::unique_lock<std::mutex> lock(mutex_);
    cv_.wait(lock, [this] { return closed_ || !queue_.empty(); });
    if (queue_.empty()) {
        return std::nullopt;
    }
    T value = std::move(queue_.front());
    queue_.pop();
    return value;
}
```

逐行解释：

- `std::optional<T>`：可能返回一个任务，也可能返回空。
- `std::unique_lock`：比 `lock_guard` 灵活，条件变量需要它。
- `cv_.wait(...)`：如果队列为空且没关闭，就睡眠等待。
- `[this] { return closed_ || !queue_.empty(); }`：唤醒条件。
- `if (queue_.empty()) return std::nullopt;`：队列关闭且无任务，返回空。
- `T value = std::move(queue_.front());`：取出队首任务。
- `queue_.pop();`：删除队首。
- `return value;`：返回任务。

ThreadPool 核心代码

```
void ThreadPool::worker_loop() {  
    while (auto task = tasks_.pop()) {  
        (*task)();  
    }  
}
```

逐行解释：

- `worker_loop()`：每个工作线程执行的循环。
- `tasks_.pop()`：从任务队列取任务。
- `while (auto task = ...)`：如果取到任务就进入循环；如果队列关闭且为空，结束循环。
- `(*task)();`：执行任务。`task` 是 optional，`*task` 取出里面的函数对象，再 `()` 调用。

理论知识：生产者消费者模型

当前项目中：

- 生产者：主线程的 `handle_read()`。
- 队列：`BlockingQueue<Task>`。
- 消费者：线程池里的工作线程。

本节练习

1. 解释为什么需要 `mutex`。
2. 解释为什么需要 `condition_variable`。
3. 解释为什么关闭队列时要 `notify_all`。

第 26 课：异步日志

本节目标

理解为什么日志要异步写，以及当前项目如何实现。

当前项目文件：

- `include/tiny_http/async_logger.h`
- `src/async_logger.cpp`

核心代码

```
void AsyncLogger::log(LogLevel level, const std::string& message) {
    if (static_cast<int>(level) < static_cast<int>(min_level_)) {
        return;
    }
    {
        std::lock_guard<std::mutex> lock(mutex_);
        if (!running_) {
            return;
        }
        queue_.push_back(format(level, message));
    }
    cv_.notify_one();
}
```

逐行解释：

- `void AsyncLogger::log(...)`：记录一条日志。
- `LogLevel level`：日志级别，例如 Info、Warn、Error。
- `const std::string& message`：日志内容。
- `static_cast<int>(level)`：把枚举转换成整数方便比较。
- `if (...) return;`：低于最小级别的日志直接丢弃。
- `{ ... }`：锁作用域。
- `std::lock_guard`：加锁保护队列。
- `if (!running_) return;`：日志器没运行就不记录。
- `queue_.push_back(format(level, message));`：格式化日志并放入队列。
- `cv_.notify_one();`：唤醒后台日志线程。

后台线程：

```
batch.swap(queue_);
for (const auto& line : batch) {
    output_ << line << '\n';
}
output_.flush();
```

逐行解释：

- `batch.swap(queue_)`：快速交换队列，把日志批量拿出来，减少持锁时间。
- `for (...)`：遍历这一批日志。
- `output_ << line << '\n';`：写入文件。
- `output_.flush();`：刷新到文件，降低丢日志概率。

理论知识：同步日志 vs 异步日志

同步日志：

业务线程直接写文件，简单但可能慢。

异步日志：

业务线程只入队，后台线程写文件，主流程更快。

缺点：

程序异常崩溃时，队列里没写完的日志可能丢失。

本节练习

1. 解释 `min_level_`。
2. 解释为什么要 `flush`。
3. 解释为什么析构函数要调用 `stop()`。

第 27 课：优雅关闭

本节目标

让服务器退出时释放资源。

main.cpp 核心代码

```
tiny_http::LinuxHttpServer* g_server = nullptr;

void handle_signal(int) {
    if (g_server != nullptr) {
        g_server->stop();
    }
}
```

逐行解释：

- `tiny_http::LinuxHttpServer* g_server = nullptr;`：全局服务器指针，初始为空。
- `void handle_signal(int)`：信号处理函数，参数是信号编号，这里不需要使用，所以省略名字。
- `if (g_server != nullptr)`：确保服务器对象存在。
- `g_server->stop();`：停止服务器。

注册信号：

```
std::signal(SIGINT, handle_signal);
std::signal(SIGTERM, handle_signal);
```

逐行解释：

- `SIGINT`：通常是按 `Ctrl+C`。
- `SIGTERM`：终止信号，常用于正常请求进程退出。
- `std::signal(...)`：注册处理函数。

stop 做什么

1. `running_ = false`
2. 停止线程池
3. 关闭所有客户端 `fd`
4. 关闭监听 `fd`
5. 关闭 `epoll fd`
6. 停止日志线程

理论知识：RAII

RAII 是 C++ 常用资源管理思想：

对象创建时获取资源。
对象销毁时释放资源。

当前项目中：

- `ThreadPool::~ThreadPool()` 调用 `stop()`。
- `AsyncLogger::~AsyncLogger()` 调用 `stop()`。
- `LinuxHttpServer::~LinuxHttpServer()` 调用 `stop()`。

本节练习

1. 解释 `Ctrl+C` 后服务器如何停止。
2. 解释为什么线程需要 `join`。
3. 解释如果日志线程不 `stop` 可能发生什么。

第 28 课：Linux 下构建、运行、测试服务器

构建

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j$(nproc)
```

逐行解释：

- `cmake -S . -B build`：生成构建文件。
- `-DCMAKE_BUILD_TYPE=Debug`：使用 Debug 构建，方便调试。
- `cmake --build build`：开始编译。
- `-j$(nproc)`：并行编译，`nproc` 会输出 CPU 核心数。

运行测试

```
ctest --test-dir build --output-on-failure
```

解释：

- 运行 CMake 注册的测试。
- 失败时打印详细输出。

运行服务器

```
./build/tiny_httpd 8080 www 4
```

参数解释：

- `8080`：监听端口。
- `www`：静态文件根目录。
- `4`：工作线程数量。

用 curl 测试

```
curl -v http://127.0.0.1:8080/
```

解释：

- `curl`：发送 HTTP 请求。
- `-v`：verbose，显示请求和响应细节。
- `127.0.0.1`：本机。
- `8080`：服务器端口。
- `/`：请求首页。

只看响应头：

```
curl -I http://127.0.0.1:8080/
```

测试 404：

```
curl -v http://127.0.0.1:8080/not-found.html
```

测试路径穿越防护：

```
curl -v http://127.0.0.1:8080/../README.md
```

查看端口占用

```
ss -lntp | grep 8080
```

解释：

- `ss`：查看 socket 状态。

- `-l`：只看监听状态。
- `-n`：数字形式显示端口。
- `-t`：TCP。
- `-p`：显示进程。
- `grep 8080`：过滤出 8080 端口。

停止服务器

如果服务器在前台运行：

```
Ctrl+C
```

如果在后台运行，先查进程：

```
ps aux | grep tiny_httpd
```

再发送终止信号：

```
kill -TERM 进程号
```

解释：

- `ps aux`：查看所有进程。
- `grep tiny_httpd`：过滤服务器进程。
- `kill -TERM`：发送正常终止信号。

第 29 课：初学者调试思路

编译错误怎么看

常见错误：

```
fatal error: tiny_http/http_parser.h: No such file or directory
```

说明：

```
头文件路径没配置好。  
检查 target_include_directories。
```

常见错误：

```
undefined reference to tiny_http::HttpParser::parse
```

说明：

声明找到了，但实现文件没有参与链接。

检查 CMakeLists.txt 有没有加入 `src/http_parser.cpp`。

运行错误怎么看

端口被占用：

```
failed to start server
```

排查：

```
ss -lntp | grep 8080
```

文件返回 404：

```
ls -la www
```

确认文件是否真的在 `www` 目录下。

添加临时日志

可以在代码中临时加：

```
std::cerr << "handle_read fd=" << fd << std::endl;
```

逐行解释：

- `std::cerr`：标准错误输出，适合打印调试信息。
- `<<`：输出内容。
- `fd`：当前连接 `fd`。
- `std::endl`：换行并刷新。

调试完记得删除或改成正式日志。

第 30 课：你应该如何复述这个项目

一句话描述

这是一个基于 C++17 和 Linux epoll 的轻量级 HTTP 服务器，支持静态资源访问、HTTP 请求解析、Keep-Alive、线程池和异步日志。

按请求流程描述

服务器启动后创建监听 socket，并注册到 epoll。

当有新连接到来时，accept4 得到非阻塞 client fd，并加入 epoll。

当 client fd 可读时，主线程循环 recv 到 EAGAIN，把数据放入连接输入缓冲区。

HTTP 解析器判断请求完整后，主线程把 raw 请求提交到线程池。

工作线程解析请求、映射静态文件路径、读取文件、生成 HTTP 响应。

响应放入连接输出缓冲区后，主线程把 fd 切换到 EPOLLOUT。

当 fd 可写时，主线程非阻塞 send，直到写完或 EAGAIN。

写完后根据 Keep-Alive 决定关闭连接或继续监听读事件。

面试官常问

1. 为什么用 epoll?
2. ET 和 LT 有什么区别?
3. 为什么 recv 要读到 EAGAIN?
4. send 一次发不完怎么办?
5. 为什么要线程池?
6. 为什么不能每个连接一个线程?
7. HTTP 解析器如何处理半包?
8. 如何防止路径穿越?
9. Keep-Alive 怎么实现?
10. 项目还有哪些可以优化?

不要夸大

当前项目没有实现：

HTTPS
HTTP/2
MySQL 连接池
sendfile 零拷贝
反向代理
完整生产级 HTTP 解析

可以说：

这些是后续扩展方向。

不要说：

我已经实现了完整 Nginx。

推荐学习节奏

如果你每天能学 1 到 2 小时：

第 1 周：
Linux 命令、Vim、CMake、HTTP 响应、HTTP 请求结构。

第 2 周：
HTTP 解析器、单元测试、MIME、静态文件。

第 3 周：
socket、非阻塞 I/O、epoll、Connection。

第 4 周：
读请求、处理请求、写响应、线程池、Keep-Alive。

第 5 周：
异步日志、优雅关闭、运行测试、调试、面试表达。

真正学会的标准不是“看完文档”，而是：

你能从 0 写出一个简化版。

你能运行服务器并用 `curl` 测试。

你能讲清楚一个请求从进入到响应出去的全过程。

你能解释每个核心模块为什么存在。